

Hands-On-Learning: Cloud-Native Security: Container Vulnerability Scanning and Security Reporting with Trivy

Learning Objective

By the end of this activity, learners will be able to:

- Install and configure Trivy scanner for comprehensive container security analysis
 - Perform vulnerability scanning on container images to identify security risks and misconfigurations
 - Generate detailed security reports in multiple formats (JSON, HTML, SARIF) for different stakeholder needs
-

- Analyze vulnerability scan results to assess severity levels and prioritize remediation efforts
 - Develop targeted mitigation strategies for container security vulnerabilities based on cloud-native best practices
 - Apply container security principles to establish secure image supply chains and runtime protection
-

Description

This practical workshop demonstrates cloud-native security practices using Trivy, an open-source vulnerability scanner designed for containers and cloud-native environments. Students will scan real container images to identify vulnerabilities, misconfigurations, and security risks, then generate professional security reports and develop remediation strategies. The lab emphasizes practical application of container security principles essential for modern cloud-native architectures.

Structure of the Activity

Tools used

Trivy (open-source vulnerability scanner by Aqua Security)

Command line interface (Linux/macOS/Windows)

Sample container images for analysis

JSON processing tools (jq) for report analysis

Scenario

Your organization is migrating to a cloud-native architecture with containerized applications running on Kubernetes. As a cloud security engineer, you've been tasked with establishing a comprehensive container security program that includes vulnerability scanning, security reporting, and remediation processes. The development teams are deploying various container images to production, and you need to assess their security posture before deployment. Your analysis will inform security policies, establish scanning baselines, and provide actionable recommendations to ensure the organization's container infrastructure follows cloud-native security best practices including zero trust principles and defense-in-depth strategies.

Objective

Use Trivy to perform comprehensive security scanning of container images, generate professional security reports for different audiences, and develop risk-based remediation strategies for cloud-native environments.



Instructions for learners

1 Install Trivy Scanner

Install Trivy using the official installation script:

```
curl -sfL  
https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/in  
stall.sh | sh -s -- -b /usr/local/bin  
# Alternative: Direct binary download  
# Visit https://github.com/aquasecurity/trivy/releases for platform-  
specific binaries
```

For Ubuntu/Debian users, you can also use apt:

```
sudo apt-get install wget apt-transport-https gnupg lsb-release  
wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |  
sudo apt-key add -  
echo deb https://aquasecurity.github.io/trivy-repo/deb $(lsb_release -  
sc) main | sudo tee -a /etc/apt/sources.list.d/trivy.list  
sudo apt-get update && sudo apt-get install trivy
```

Verify installation and check available commands:

```
trivy --version  
trivy --help
```

2 Pull Sample Container Images for Analysis

Pull various container images representing different security postures:

```
docker pull vulnerables/web-dvwa:latest  
docker pull nginx:1.20  
docker pull node:14-alpine
```

Verify images are available locally:

```
docker images
```

Note: If Docker is not available, Trivy can scan images directly from registries

3 Perform Comprehensive Vulnerability Scans

Start with a basic vulnerability scan:

```
trivy image vulnerables/web-dvwa:latest
```

Perform targeted severity scanning:

```
trivy image --severity CRITICAL,HIGH vulnerables/web-dvwa:latest  
trivy image --severity UNKNOWN,LOW,MEDIUM,HIGH,CRITICAL  
nginx:1.20
```

Scan for multiple security issues:

`trivy image --scanners vuln,misconfig,secret node:14-alpine`

Compare different base images:

`trivy image --severity HIGH,CRITICAL alpine:3.14`

`trivy image --severity HIGH,CRITICAL ubuntu:20.04`

4 Generate Professional Security Reports

Create JSON report for programmatic analysis:

```
trivy image --format json --output dvwa-security-report.json  
vulnerables/web-dvwa:latest  
trivy image --scanners vuln,misconfig,secret --format json --output  
nginx-full-report.json nginx:1.20
```

Create HTML report for stakeholders:

```
wget https://raw.githubusercontent.com/aquasecurity/trivy/main/  
contrib/html.tpl  
trivy image --format template --template html.tpl --output security-  
report.html vulnerables/web-dvwa:latest
```

Generate SARIF report for CI/CD integration:

```
Wget https://raw.githubusercontent.com/aquasecurity/trivy/main/contrib/sarif.tpl  
trivy image --format template --template sarif.tpl --output security-report.sarif nginx:1.20
```

Create summary reports by severity:

```
trivy image --severity CRITICAL --format json --output critical-issues.json vulnerables/web-dvwa:latest
```

5 Analyze Security Report Data

Analyze the JSON report using command-line tools:

```
cat dvwa-security-report.json | jq '[.Results[].Vulnerabilities[]? |  
.Severity] | group_by(.) | map({severity: .[0], count: length})'  
cat dvwa-security-report.json | jq '.Results[].Vulnerabilities[]? |  
select(.Severity == "CRITICAL") | {ID: .VulnerabilityID, Package:  
.PkgName, Version: .InstalledVersion, Title: .Title}'  
cat dvwa-security-report.json | jq '.Results[].Vulnerabilities[]? |  
select(.FixedVersion != "") | {ID: .VulnerabilityID, Package: .PkgName,  
Current: .InstalledVersion, Fixed: .FixedVersion}'
```


Extract key metrics for reporting:

```
cat dvwa-security-report.json | jq '[.Results[].Vulnerabilities[]?] | length'  
cat dvwa-security-report.json | jq  
'[.Results[].Vulnerabilities[]?..PkgName] | unique | length'
```

Review HTML report in a web browser to see visualized results

6 Develop Targeted Mitigation Strategies

Based on your scan results, create a comprehensive mitigation plan:
Immediate Actions (Critical/High Severity):

Document all critical vulnerabilities requiring immediate patching

Identify base image updates available (e.g., newer Alpine, Ubuntu versions)

List packages with security patches available

Prioritize vulnerabilities affecting internet-facing containers

Short-term Security Improvements:

Implement automated vulnerability scanning in CI/CD pipelines

Establish image approval processes based on vulnerability thresholds

Create security policies for acceptable risk levels

Plan regular base image update cycles

Long-term Cloud-Native Security Strategy:

Implement runtime security monitoring for deployed containers

Establish secure image supply chain with signed images

Deploy network policies and micro-segmentation

Implement secrets management and encryption at rest/transit

Create a risk assessment matrix:

Risk Level = Severity × Exposure × Exploitability

Consider CVSS scores, public exploit availability, and container exposure

Prioritize fixes based on business impact and remediation complexity

Organize Information Following This Structure for Peer Review

1. Title Page:

Project title: "Cloud-Native Security Assessment: Container Vulnerability Analysis and Remediation" Your name, date, container images analyzed

2. Requirement 1: SBOM Generation and Inventory Analysis

Provide vulnerability statistics for each scanned container image

Compare security postures between different base images (Alpine vs Ubuntu, etc.)

Highlight critical and high-severity vulnerabilities requiring immediate attention

Include screenshots of scan outputs and generated reports

Document any misconfigurations or exposed secrets discovered

3. Requirement 2: Security Report Generation and Analysis

Demonstrate proficiency with multiple report formats (JSON, HTML, SARIF)

Show analysis of JSON data using command-line tools

Present vulnerability trends and patterns across different images

Include sample sections from generated HTML reports

Explain the business value of different report formats for various stakeholders

4. Requirement 3: Cloud-Native Security Recommendations and Mitigation Strategy

Present a prioritized remediation roadmap with specific timelines

Recommend secure base images and image hardening techniques

Propose CI/CD pipeline integration for continuous security scanning

Suggest runtime security controls and monitoring strategies

Include compliance considerations and policy recommendations for cloud-native environments

Instructions for Documenting and Sharing the Project for Peer Review



1. Document the Project

Use word processing software to create your report.

Insert the diagram, map, screenshot, or image directly into your Word document, if available.

Ensure all sections of the project document structure are completed.

Proofread and edit your report for clarity and accuracy.



Share the Project

Save your report and presentation in PDF format.

Upload your documents to the “Peer Review” area in the course shell.

Provide a brief description of your project in the submission post.



Peer Review

Review the projects submitted by your peers.

Provide constructive feedback and comments on their work.

Additional Resources and References:

Trivy Documentation: <https://aquasecurity.github.io/trivy/>

Container Security Best Practices:
<https://kubernetes.io/docs/concepts/security/>

NIST Container Security Guidelines:
<https://csrc.nist.gov/publications/detail/sp/800-190/final>

CIS Docker Benchmark: <https://www.cisecurity.org/benchmark/docker>

OWASP Container Security Top 10: <https://owasp.org/www-project-container-top-10/>

Cloud Native Security Whitepaper: <https://github.com/cncf/sig-security/blob/main/security-whitepaper/cloud-native-security-whitepaper.md>